1. Check whether something is possible, instead of what would it take to do this well

2. Try to understand it from the perspective of a delivery engineer
   how will they likely try to use it and what are the consequences

3. Without a use case, its probably not the right setup to make a choice.

What I've learned

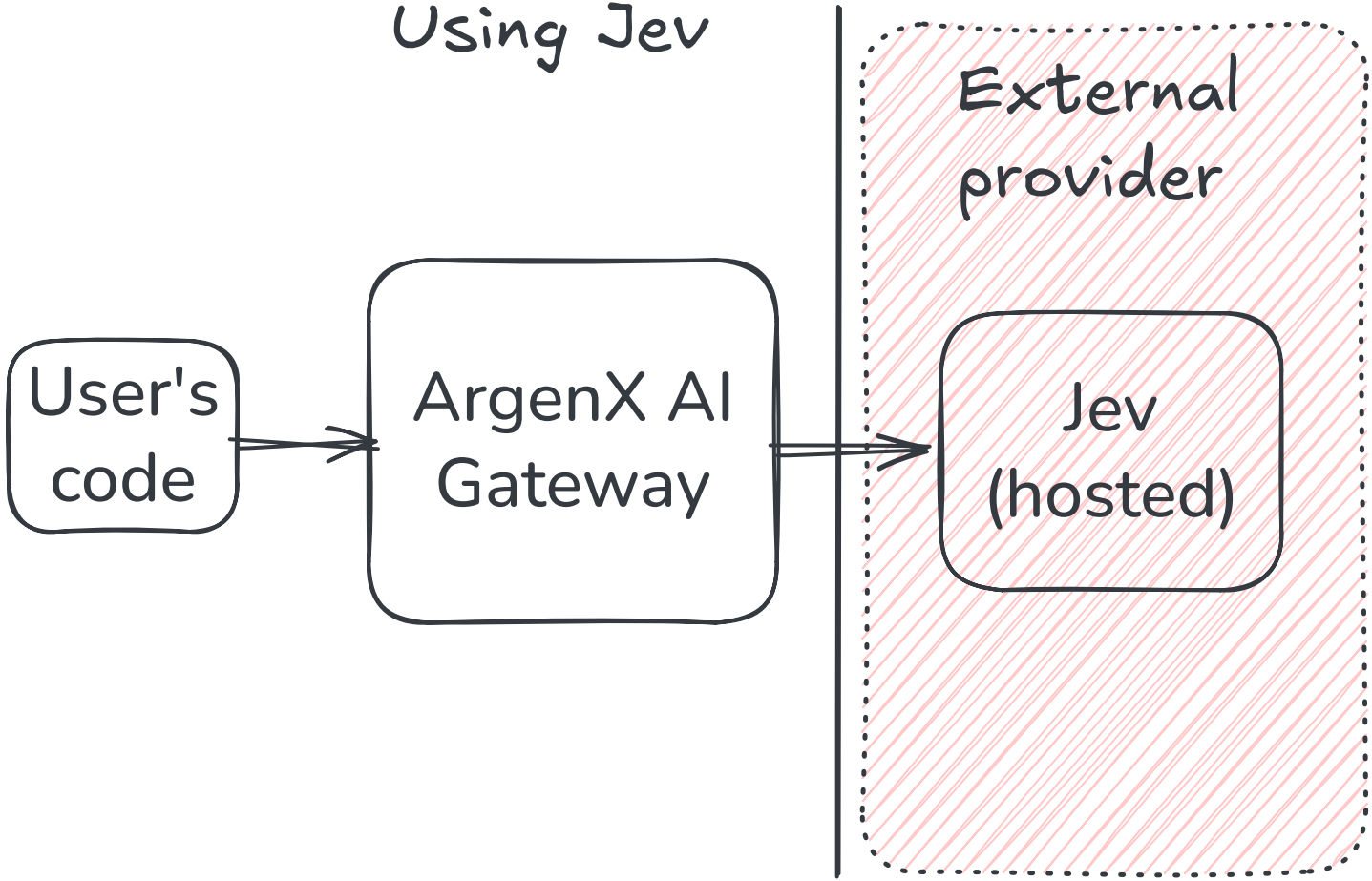
A judgment model - a fast model that:
given a text (up to 32000 tokens Jev, and 512 tokens Laya)
and a question (another several words that fit within the token limit)
responds with a probability between multiple answers (up to 20)

Q.: Can we run these?

Jev

Benefit: larger context, more trained data out of the box

Consuming from hosted service
(The same way we do with other LLM models)


Using Jev
External provider
User's code
ArgenX AI Gateway
Jev (hosted)

With Laya - the open weight model

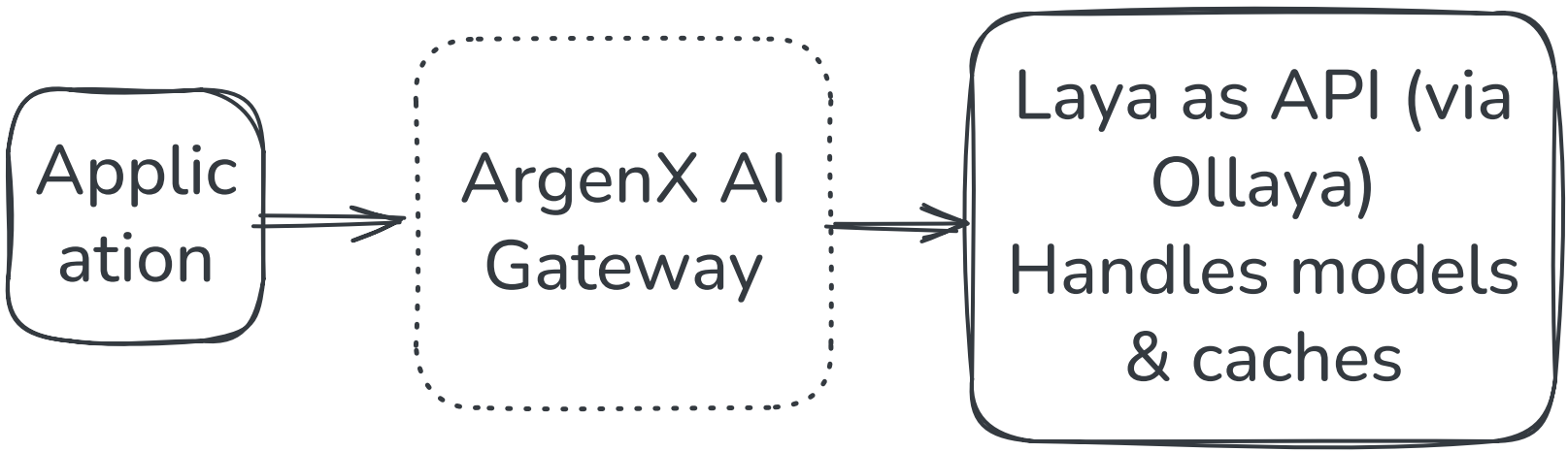
Benefit: own infrastructure, can be trained on own data

* Running it embedded with the application
* Running it as an API sidecar
* Running it on the platform and consuming it from there

Running Embedded

Use case application calling the embedded lib (~1GB model download, or in the container)

Running as API



Fine Tuning Laya

Laya is not working well with specialty texts

It can be fine tuned

even in a docker container, using only CPU, 8GB RAM